

Cooperative Concurrency

Formalization and Implementation via Algebraic Effects and Handlers

Raphael Frank

Programming Language Theory
Institute of Science Tokyo

1 Introduction

Concurrency and multithreading have become an existential feature in modern programming languages [4]. Concurrency refers to a system being able to manage and execute multiple computations seemingly simultaneously or in overlapping time intervals [4]. These features enable software developers to build powerful and efficient software by making full use of parallel architecture in computers. Parallel computations also allow for strong types of parallel algorithms. All in all, concurrency enhances several programming languages today with a richer feature set. With the recent upcoming of algebraic effects and handlers [7], we have yet another rich feature that can enhance programming languages.

In this report, we look into cooperative concurrency by formalizing an approach for our *BoolInt* language and by looking into practical implementations that make use of algebraic effects and handlers. Cooperative concurrency is a simplified case of concurrency where several threads run in parallel, but only one at a time [1]. At the end of this report, we give an outlook on first class asynchronous effects [2] that oppose the synchronous nature of normal algebraic effects while also discussing their potential use in modern programming languages.

Modern programming languages have a rich tool set to make use of parallelism [5]. However, we simplify our requirements to the `fork` and `yield` functions. The `fork` function forks the current thread, starting a new one with a given expression while keeping the current thread. If the current thread finishes execution or executes the `yield` function, it gives the execution to the scheduler which schedules a new thread and continues its execution. By reducing our tool set to the minimum functions needed to simulate concurrency, we can concentrate on the underlying system more easily.

2 Implementation

To see how cooperative concurrency can be achieved in a functional programming language using algebraic effects and handlers, we look at two examples.

2.1 Pyret Implementation

The first example is based on a simple implementation by Bauer et. al [3] that we write in the Pyret programming language. The goal is to use algebraic effects and handlers to easily store/continue continuations of different threads. Pyret is chosen due to the familiarity for most of the other students.

First, we add the effect `Coop` which defines the two operations `t_yield` and `t_fork`. The `fork` operation takes a function `f` as an argument, that will be executed as a new thread. The `yield` function has no parameter and can therefore be executed anytime inside of a thread. Both operations return a simple non-meaningful `Unit` value.

```

effect Coop:
  | t_yield() -> Unit
  | t_fork(f :: (Unit -> Unit)) -> Unit
end

```

After that, we can define the handler. In order to make it easier to alter the handling of the operations depending on the current threads, we define a function that implements the handler. The function mainly defines the scheduler that manages the threads. For simplicity, we implement a RoundRobin scheduler [8] that executes the first thread in a list and moves it to the end of the list in case that it yields. The function works recursively with a list of lambda functions - that represents the open threads - as a parameter. We can then perform a case analysis with a custom handler to handle occurring thread operations¹.

The function begins by executing the first thread in the list. If it runs through completely and finishes, the function continues with the rest of the threads. If an operation occurs during execution, the function handles them accordingly. For a yield, we put the current continuation at the end of the thread list and schedule the next round. For a fork, we put the forked expression at the beginning of the list and the rest (i.e. continuation) of the current execution at the end of the list.

```

fun RoundRobin(threads):
  cases(List) threads:
    | empty => unit
    | link(t, ts) => block:
      handle:
        block:
          t()
          RoundRobin(ts)
        end
      with handler(Coop):
        | t_yield(), k => RoundRobin(ts.append([list: k]))
        | t_fork(f), k => RoundRobin([list: f].append(ts.append([list: k])))
      end
    end
  end
end
end

```

Now, we can use this scheduler to execute arbitrary functions that contain thread operations. The following code shows an example usage of two simple functions with one fork and one yield. The executed function forks the second function. The second function then yields after starting so that the first function gets scheduled next and finishes. Note, that we have to call the scheduler with the function inside of a list in order to work properly. It would also be possible to define a helper function that just takes in one function and builds the list automatically.

```

fun baa():
  block:
    print("baa started!")
    t_yield()
    print("baa ended!")
  end
end

```

¹ Note, that in our version of Pyret, there is a bug that the matched thread-list `t` and `ts` don't get substituted in the handler.

```

fun foo():
  block:
    print("foo started!")
    t_fork(baa)
    print("foo ended!")
  end
end

```

```
RoundRobin([list: foo])
```

Further extensions of our implementation could add different schedulers or even more thread operations to make our library more versatile.

2.2 OCaml Implementation

The second example discusses the official implementation of cooperative concurrency via algebraic effects and handlers in the OCaml programming language. The complete code originates from the OCaml effects tutorial [9].

The operations and the scheduler function are defined in a module. The definition is pretty similar to our Pyret version. The main difference is the naming of the fork function to `async`.

```

module type Scheduler = sig
  val async : (unit -> 'a) -> unit
  (** [async f] runs [f] concurrently *)
  val yield : unit -> unit
  (** yields control to another task *)
  val run    : (unit -> 'a) -> unit
  (** Runs the scheduler *)
end

```

Another way of managing the open threads is a mutable queue. The OCaml module manages this queue via enqueueing and dequeuing the current threads. This method makes managing the queue more versatile than our approach with a mutable list that is just changed in the recursion step. In the following code, the queue for storing the threads is defined along with helper functions.

```

let q = Queue.create ()
let enqueue t = Queue.push t q
let dequeue () =
  if Queue.is_empty q then ()
  else Queue.pop q ()

```

Lastly, we want to describe the differences that occur in the scheduler/running function with the different design decisions in place. Due to the versatile queue as a thread storage, enqueue and dequeue operations can easily be performed multiple times in a block to modify the queue. Due to OCaml's rich effect tool set, we can handle occurring exceptions (here, we just propagate the exception) and especially what happens when a thread finishes/returns (in `retc`).

```

let rec run : 'a. (unit -> 'a) -> unit =
fun main ->
  match_with main ()
  { retc = (fun _ -> dequeue ());
    exnc = (fun e -> raise e);
    effc = (fun (type b) (eff: b Effect.t) ->

```

```

match eff with
| Async f -> Some (fun (k: (b, -) continuation) ->
    enqueue (continue k);
    run f
)
| Yield -> Some (fun k ->
    enqueue (continue k);
    dequeue ()
)
| _ -> None
})

```

In general, the OCaml implementation can implement concurrency features more smoothly due to a richer syntax for effects (e.g. a return handler). However, we were also able to implement a scheduler with nearly the same functionality in regards to cooperative concurrency. We therefore conclude that algebraic effects and handlers are very suited for an implementation of cooperative concurrency due to their ability to easily work with continuations. We also conclude that Pyret is a great language to give students simple but effective tools to implement such features.

3 Formalization

In the last section, we saw that we could use algebraic effects to implement cooperative concurrency. In this section, we want to look into simple ways to formalize concurrency operations to fit into our `BoolInt` language with effects. Since we have already added effects to our language, our idea was to add concurrency features as first class operations instead of implementing them just with effects. First, we try to add our simple fork and yield operations as well as the already introduced RoundRobin scheduler. Afterwards, we give an intuition of how to extend that system with channels [2] that threads can use to exchange data with each other. Note, that the implementation as well as the definition is highly depending on different design decisions. We chose implementations based on simplicity and intuition about effectiveness.

3.1 Simple Operations and Scheduler

For the addition of simple concurrency operations, we orient ourselves to the structure of the course material. Therefore, we first define the general syntax. Then, we extend the abstract machine semantics (AMS). We decided to concentrate on the AMS instead of the reduction semantics since we find it more convenient to handle multiple expressions with AMS. Lastly, we will define the typing rules.

Syntax We represent an operation on cooperative concurrency as a combination of the operation c and the rest of the expression e separated by a semicolon. This way, an expression can fork or yield at any time while also being able to evaluate to a meaningful value. Possible thread operations are fork and yield. The fork operation takes an expression that will be evaluated in the new thread.

Expressions $e ::= \dots$	
$c; e$	ThreadOPCall
ThreadOPCall $c ::= \text{fork } e$	ForkCall
yield	YieldCall

An example for a syntactically well written expression would be the following which we will also use as a general example when evaluating:

`fork add1 1 ; add1 yield ; 2`

The example expression is well formed as it is, but for better reading, we could add parentheses:

`(fork (add1 1) ; (add1 yield ; 2))`

Abstract Machine Semantics Since we handle multiple expressions that each evaluate to a value, we want to give out a list of these values at the end of our computation. We introduce a list of values as a possible machine value.

Machine values $v ::= \dots$	l	ValueList
ValueList	$l ::= []$	Empty value list
	$v :: l$	Non-empty value list

We add an additional configuration mode “sched” for handling our scheduling. In order to properly manage the state of the current computation of a thread, we save a current state (or configuration) as a “pure” configuration. A pure configuration consists of the current expression e and the current continuation stack K of an evaluation. Because the scheduler takes care of setting configurations up in the correct mode, we don’t have to save the configuration mode in pure configurations. These pure configurations are then saved in a list T that is carried by the abstract machine. In “eval” and “cont” mode, T is only storing the configurations of every other thread. During “sched” mode, T stores all configurations. Since “sched” mode doesn’t compute/evaluate on any configuration directly, it is sufficient that it only holds the thread list.

Pure configurations $C_p ::= \langle e, K \rangle$
Configurations $C ::= \langle e, K, T \rangle_{\text{eval}}$
$\langle K, v, T \rangle_{\text{cont}}$
$\langle T \rangle_{\text{sched}}$
Configuration list $T ::= [] \mid C_p :: T$

At the start of evaluation, there are no other threads. Therefore, we initialize with an empty configuration list.

initialization $e \rightarrow \langle e, [], [] \rangle_{\text{eval}}$

In general, transition rules apply as if the configuration list doesn’t exist (i.e. it preserves the list). The following is an example that applies this rule.

M-Eval-Val $\langle v, K, T \rangle_{\text{eval}} \rightarrow \langle K, v, T \rangle_{\text{cont}}$

We can then add additional transition rules. Note, that theses rules depend on the scheduler you want to implement. Fork adds another configuration with the new expression to the beginning of the list and continues the current evaluation (however, directly changing into scheduling would also be an alternative). Yield captures the current evaluation, adds its configuration to the beginning of the list and switches to schedule mode. When the current expression finishes evaluation by evaluation to a value, we switch to schedule mode to continue not finished evaluations. When scheduling, we want to put the most recent configuration (i.e. the one at the

start of T) to the end of the list and continue evaluation the next configuration. Note that the schedule might also move items or list parts that don't exist because of the notation (e.g. when there is only one thread). In other words, it just puts the first element to the last position and then evaluates the first item in the list (whatever it happens to be).

$$\begin{array}{ll}
\text{M-Eval-Fork} & \langle \text{fork } e'; e, K, T \rangle_{\text{eval}} \rightarrow \langle e, K, \langle e', [] \rangle :: T \rangle_{\text{eval}} \\
\text{M-Eval-Yield} & \langle \text{yield}; e, K, T \rangle_{\text{eval}} \rightarrow \langle \langle e, K \rangle :: T \rangle_{\text{sched}} \\
\text{M-Cont-Val} & \langle [], v, T \rangle_{\text{cont}} \rightarrow \langle \langle v, [] \rangle :: T \rangle_{\text{sched}} \\
\text{M-Sched-Expr} & \langle t :: \langle e, K \rangle :: T :: [] \rangle_{\text{sched}} \rightarrow \langle e, K, T :: t :: [] \rangle_{\text{eval}} \\
& \text{if } e \neq v
\end{array}$$

We add the extra constrain that continuing a configuration requires it to be an expression rather than a value. It is not necessary to have this constrain and the semantics would still work. However, we want to reduce some unnecessary evaluation steps (i.e. switching to evaluation a value, switching to cont mode with that value and switching to schedule mode again) so we add another transition rule that lets us simply skip over already finished evaluations.

$$\begin{array}{ll}
\text{M-Sched-Val} & \langle t :: \langle v, [] \rangle :: T :: [] \rangle_{\text{sched}} \rightarrow \langle \langle v, [] \rangle :: T :: t :: [] \rangle_{\text{sched}} \\
& \text{if } \exists c \in T: c \neq \langle v, [] \rangle
\end{array}$$

We consider the computation to be complete if all threads finished evaluation. In other words, we finalize if every configuration in T consists of a value with an empty continuation stack. We finish by collecting every evaluated value in a list (this was the reason to add a value list as a possible machine value).

$$\text{finalization} \quad \langle T \rangle_{\text{sched}} \rightarrow \langle \langle v_1, [] \rangle :: \dots \langle v_n, [] \rangle :: [] \mapsto v_1 :: \dots v_n :: [] \rangle$$

The following computation shows how our example expression is being evaluated using the new abstract machine semantics.

$$\begin{array}{ll}
\text{fork add1 1; add1 yield; 2} & \\
\rightarrow \langle \text{fork add1 1; add1 yield; 2}, [], [] \rangle_{\text{eval}} & \text{initialization} \\
\rightarrow \langle \text{add1 yield; 2}, [], \langle \text{add1 1}, [] \rangle :: [] \rangle_{\text{eval}} & \text{by M-Eval-Fork} \\
\rightarrow \langle \text{yield; 2}, \text{add1} :: [], \langle \text{add1 1}, [] \rangle :: [] \rangle_{\text{eval}} & \text{by M-Eval-Add1} \\
\rightarrow \langle \langle 2, \text{add1} :: [] \rangle :: \langle \text{add1 1}, [] \rangle :: [] \rangle_{\text{sched}} & \text{by M-Eval-Yield} \\
\rightarrow \langle \text{add1 1}, [], \langle 2, \text{add1} :: [] \rangle :: [] \rangle_{\text{eval}} & \text{by M-Sched-Expr} \\
\rightarrow \langle 1, \text{add1} :: [], \langle 2, \text{add1} :: [] \rangle :: [] \rangle_{\text{eval}} & \text{by M-Eval-Add1} \\
\rightarrow \langle \text{add1} :: [], 1, \langle 2, \text{add1} :: [] \rangle :: [] \rangle_{\text{cont}} & \text{by M-Eval-Val} \\
\rightarrow \langle [], 2, \langle 2, \text{add1} :: [] \rangle :: [] \rangle_{\text{cont}} & \text{by M-Cont-Add1} \\
\rightarrow \langle \langle 2, [] \rangle :: \langle 2, \text{add1} :: [] \rangle :: [] \rangle_{\text{sched}} & \text{by M-Cont-Val} \\
\rightarrow \langle 2, \text{add1} :: [], \langle 2, [] \rangle :: [] \rangle_{\text{eval}} & \text{by M-Sched-Expr} \\
\rightarrow \langle \text{add1} :: [], 2, \langle 2, [] \rangle :: [] \rangle_{\text{cont}} & \text{by M-Eval-Val} \\
\rightarrow \langle [], 3, \langle 2, [] \rangle :: [] \rangle_{\text{cont}} & \text{by M-Cont-Add1} \\
\rightarrow \langle \langle 3, [] \rangle :: \langle 2, [] \rangle :: [] \rangle_{\text{sched}} & \text{by M-Cont-Val} \\
3 :: 2 :: [] & \text{finalization}
\end{array}$$

To give more context to the final values, we could also add labels to forks (that they carry into the final value list) so that we can identify the resulting values more easier. However, for simplicity, we didn't look into this idea more deeply.

Type System For the type system, we have to define the typing rule for each operation. Here, we simplify a lot by assuming a whole fork expression to accept the type of the parent expression (i.e. not the expression inside the fork). This way, we don't have to deal with collecting the type of every forked expression in a grouped list and just expect all forked expression to simply be well typed. Also, we grant the expression inside the fork to have an arbitrary effect. It would also be possible to expect the forked expression to have the same effect as its parent, but we decided that it makes more sense that it has to define its own handlers since it runs with a different continuation. Since the yield operation doesn't change to following evaluation of the expression, it propagates the required type to e .

$$\frac{\Gamma \vdash e' : \tau' ! \varepsilon' \quad \Gamma \vdash e : \tau ! \varepsilon}{\Gamma \vdash \text{fork } e' ; e : \tau ! \varepsilon} \text{ T-Fork}$$

$$\frac{\Gamma \vdash e : \tau ! \varepsilon}{\Gamma \vdash \text{yield} ; e : \tau ! \varepsilon} \text{ T-Yield}$$

Now we look at the typing derivation of our example expression.

$$\frac{\frac{\frac{}{\emptyset \vdash 1 : \text{int} ! \{\}} \text{ T-Int}}{\emptyset \vdash \text{add1 } 1 : \text{int} ! \{\}} \text{ T-Add1} \quad \frac{\frac{\frac{}{\emptyset \vdash 2 : \text{int} ! \{\}} \text{ T-Int}}{\emptyset \vdash \text{yield} ; 2 : \text{int} ! \{\}} \text{ T-Yield}}{\emptyset \vdash \text{add1 yield} ; 2 : \text{int} ! \{\}} \text{ T-Add1}}{\emptyset \vdash \text{fork add1 } 1 ; \text{add1 yield} ; 2 : \text{int} ! \{\}} \text{ T-Fork}$$

3.2 Thoughts on Extension via Channels

Our current implementation supports an expression to fork and yield while the evaluation of all threads eventually give us the final values. However, in reality, this could also be achieved by just evaluation every expression sequentially and collecting the final values manually in a list. One way of enhancing our tool set to make more use of concurrency is to add channels [2]. In this subsection, we want to give a broad idea over how to include channels in our formalization. Due to complexity, we don't formalize it completely and just give an intuition.

Via channels, threads are able to exchange data (i.e. values) with each other. They can either send to or receive from a named channel. We could add only one global channel, but we decided to go with named channels (via labels) so that threads could better make use of it and not accidentally overwrite too much data. As already said, the channels would be addressed via a simple mapping from labels. We would have a syntax that goes as the following.

`send l e      receive l`

The send expression would evaluate the expression e and overwrite the data of the channel named l with the resulting value. Furthermore, the receive expression would read out the current data that the channel named l holds and substitutes itself with that value. If the channel doesn't hold a value yet, it would block/yield the current thread.

For the abstract machine semantics, we would add a set of label to value mappings that is managed by the machine and functions as a storage buffer for the channels. We have already included yield mechanics in the previous subsection that we could reuse.

For the typing rules, send expressions would just adapt the type of its inner expression since it evaluates to it in the end. However, we would have to somehow keep track globally over the types of channels. Channel types would have to be fixed for their label in order for us to statically type-check the expression. You could do that in a similar way that effects check for

types of handlers.

In general, the introduction of channels would introduce two side effects to our computations.

1. Since the outcome of the receive expression depends on the current value of the channel, which depends on the order of threads sending to it, the resulting value is nondeterministic. However, since we use cooperative concurrency [1] with the same transition rules and deterministic scheduler, our reduction would be deterministic.
2. If a thread waits for a channel to receive data and there is no thread that will send a value, the thread will get stuck. In general, we can ensure that every channel that is read from is going to receive a value via our type system, but if two threads wait for each other without sending first (which we don't check with our type system), a deadlock [6] occurs and computation will get stuck.

4 Outlook: Asynchronous Effects

Algebraic effects that we used are inherently synchronous. If an operation occurs, the current computation gets interrupted and the corresponding handler gets executed by passing arguments and continuations to it. However, to make better use of effects in a concurrent environment (e.g. if we have multiple threads), we can modify effects to operate asynchronously. Ahman et al. [2] introduce asynchronous effects in their paper as a modified version of algebraic effects that introduces exactly that. Since the topic is quite deep and complex, we only want to give a short intuition in context of our work.

They change it so that an operation doesn't mean that it is executed immediately. An operation only signals that it needs to be executed. There, they differentiate between three phases: Signal (signaling the occurrence of the operation), Processing (asynchronous execution of the handler) and Interrupt (interruption of current computation to react to the outcome). Controlled synchronization will still be possible using an **await** operator that blocks the current computation to wait for the promised operation outcome (i.e. the promise).

To formalize their changes properly, they introduced a modified lambda calculus called the core calculus λ_{ae} . They separate the definitions into a sequential and a parallel part. In the sequential part, they extend the basic lambda calculus by signals and interrupt streams. The parallel part adds functionality to start new processes via **run** M that can then be arranged via parallel composition. There, signals can enter other processes as interrupts via different broadcasting rules. This system allows for asynchronous messaging between different processes.

5 Conclusion

With the stagnation of the increase in single core processing power, parallelism becomes more and more necessary if developers want to increase the overall computing power. In this report, we saw that algebraic effects and handlers can become very handy for implementing such systems. Especially functional programming languages can make use of continuations to give developers highly individual tools in the context of concurrency. Furthermore, a formalization of those tools can be achieved by introducing new concepts, even if it means that some formalization can become more complicated by that. In general, we think that algebraic effects and their continuations have a very high potential in concurrency systems and should be more investigated in that context and in research.

References

1. Abadi, M., Plotkin, G.D.: A model of cooperative threads. *Logical Methods in Computer Science* **Volume 6, Issue 4** (Oct 2010). [https://doi.org/10.2168/lmcs-6\(4:2\)2010](https://doi.org/10.2168/lmcs-6(4:2)2010), [http://dx.doi.org/10.2168/LMCS-6\(4:2\)2010](http://dx.doi.org/10.2168/LMCS-6(4:2)2010)
2. Ahman, D., Pretnar, M.: Asynchronous effects. *Proceedings of the ACM on Programming Languages* **5**(POPL), 1–28 (Jan 2021). <https://doi.org/10.1145/3434305>, <http://dx.doi.org/10.1145/3434305>
3. Bauer, A., Pretnar, M.: Programming with algebraic effects and handlers. *Journal of Logical and Algebraic Methods in Programming* **84**(1), 108–123 (Jan 2015). <https://doi.org/10.1016/j.jlamp.2014.02.001>, <http://dx.doi.org/10.1016/j.jlamp.2014.02.001>
4. Crafa, S.: The role of concurrency in an evolutionary view of programming abstractions. *Journal of Logical and Algebraic Methods in Programming* **84**(6), 732–741 (2015). <https://doi.org/https://doi.org/10.1016/j.jlamp.2015.07.006>, <https://www.sciencedirect.com/science/article/pii/S2352220815000784>, “Special Issue on Open Problems in Concurrency Theory”
5. Diaz, J., Muñoz-Caro, C., Niño, A.: A survey of parallel programming models and tools in the multi and many-core era. *IEEE Transactions on Parallel and Distributed Systems* **23**(8), 1369–1386 (2012). <https://doi.org/10.1109/TPDS.2011.308>
6. Hoare, C.A.R.: Communicating sequential processes. *Commun. ACM* **21**(8), 666–677 (Aug 1978). <https://doi.org/10.1145/359576.359585>, <https://doi.org/10.1145/359576.359585>
7. Kawamata, F., Unno, H., Sekiyama, T., Terauchi, T.: Answer refinement modification: Refinement type system for algebraic effects and handlers (2023), <https://arxiv.org/abs/2307.15463>
8. Matsumoto, T.: Competitive analysis of the round robin algorithm. In: Ibaraki, T., Inagaki, Y., Iwama, K., Nishizeki, T., Yamashita, M. (eds.) *Algorithms and Computation*. pp. 71–77. Springer Berlin Heidelberg, Berlin, Heidelberg (1992)
9. White, L., Dolan, S., Sivaramakrishnan, K.: Ocaml effects tutorial. <https://github.com/ocaml-labs/ocaml-effects-tutorial> (2022), accessed: 2025-08-03